Artifact: Chronos: Scalable and Cost-Effective Cloud-based RAN Emulation System

1 Artifact overview

This artifact accompanies the paper *Chronos: Scalable and Cost-Effective Cloud-based RAN Emulation System*. Its purpose is to make the implementation available for inspection, provide the scripts used to deploy Chronos, provide the paper's plot outputs and their underlying data, and demonstrate that the Chronos code is functional on a Powder/Emulab deployment.

The evaluation combines two complementary forms of evidence. First, the artifact supplies the data and scripts used to generate the plots in the paper. Second, the hosted Powder deployment gives evaluators a hands-on view of Chronos operating end to end: its components start, coordinate RAN slots, attach a UE, and carry data-plane traffic.

The repository includes `chronos-artifact.mp4`, a video tutorial covering the complete artifact and Powder evaluation workflow.

1.1 Badge scope

The artifact is prepared to support all three requested badges:

- **Artifacts Available:** the source, deployment automation, experiment data, and plotting scripts are publicly available in the GitHub repository.
- **Artifacts Evaluated–Functional:** evaluators connect to the hosted Powder deployment and verify Chronos's end-to-end operation.
- **Results Reproduced:** evaluators run the plotting notebook and reproduce the key scalability result on Powder by measuring dilation at increasing gNB/UE scale.

1.2 Obtaining the artifact

Clone the artifact repository from GitHub and enter its root directory:

```
git clone https://github.com/netsys-edinburgh/EmuRAN-Draft-Artifact.git
cd EmuRAN-Draft-Artifact
```

The repository is available at Chronos artifact GitHub repository. All installation and notebook commands below should be run from this directory.

2 Artifact components

The artifact has four main components.

2.1 Source code

The `code/` directory contains the implementation of Chronos:

Path	Purpose
`code/supervisor/`	Patched Linux 5.15/KVM source. The virtualized TSC allows guest time to be advanced or paused independently of wall-clock time.
`code/slot_monitor/`	The `custom_tsc` kernel module, shared-memory support, and slot-monitor utilities used to control virtual time.
`code/middlebox/`	Modified multi-UE nFAPI proxy. It forwards traffic, tracks local slot completion, and communicates with the global slot coordinator.
`code/gnb/`	Modified OpenAirInterface gNB source, operated in nFAPI VNF/emulated-L1 mode.
`code/ue/`	Modified OpenAirInterface UE source, operated in standalone nFAPI PNF/emulated-L1 mode.

Together these components implement PHY bypass, virtual time, slot-level synchronization, and distributed control/data-plane forwarding. The gNB, UE, and supervisor directories are complete source trees rather than small patch sets so that the evaluated implementation is preserved in full.

2.2 Deployment scripts

The `deployment/` directory contains the automation used to prepare and run Chronos:

- `deployment/provisioning/profile.py` is an Emulab/Powder profile that allocates controller/compute nodes, proxy nodes, and a global slot coordinator.
- `deployment/provisioning/scripts/` configures Ubuntu hosts, builds and installs the patched kernel, creates nested KVM guests, installs the virtual-time module and services, configures networking, and constructs the Kubernetes cluster.
- `deployment/helm/` contains Helm charts for the mobile core, global slot coordinator, proxy, gNB, UE, and persistent-UE components.
- `deployment/helm/values.yaml` controls topology size, image names, addressing, startup behavior, and the UE data-plane workload.
- Operational scripts under `deployment/helm/` start UEs, inspect or restart components, and collect logs.

These scripts perform privileged operations and modify kernels, networking, storage, VMs, and cluster state. They must only be used on disposable Powder/Emulab experiment nodes.

2.3 Plot outputs

The `results/` directory has two subdirectories:

- `results/data/` contains raw logs, packet captures, CSV/JSON intermediates, and analysis/plotting scripts from the paper experiments.
- `results/output/` contains the paper's prepared PDF plot outputs.

The top-level `plot.ipynb` provides a convenient way to inspect the plot data, plotting scripts, and PDF outputs. Evaluators may browse the prepared outputs directly and trace each result back to its associated analysis code and recorded measurements.

2.4 Running the code and deployment scripts on Powder/Emulab

The authors run a functional deployment created with the supplied Powder/Emulab profile and deployment automation. Evaluators receive SSH access to this running deployment and inspect its operation. The goal is to demonstrate that the implementation and deployment scripts work together. The preserved experiment data and outputs complement this live demonstration with evidence from the paper's full-scale evaluations.

3 Running the plotting notebook

The top-level `plot.ipynb` runs the plotting script associated with each paper figure. Each code cell handles one figure or panel, reads its inputs from `results/data/`, writes a newly generated PDF to `results/output/`, and displays that PDF inline. The notebook uses paths relative to the artifact root and therefore does not require any machine-specific path changes.

3.1 Software dependencies

The notebook runs on Linux and macOS with Python 3.10 or newer. On Linux, install Python 3 and its `venv` support using the distribution package manager. On macOS, use a current Python installation from python.org or Homebrew. The notebook and plotting scripts require the following Python packages:

- `jupyter`, `IPython`, and `ipykernel` for the notebook interface, inline display, and the dedicated artifact kernel;
- `numpy` for numerical processing;
- `pandas` for CSV and tabular data processing;
- `matplotlib` and `seaborn` for plotting; and
- `pymupdf` for rendering each generated PDF inside the notebook.

No GPU, LaTeX installation, or privileged access is needed for this notebook on either platform. Package requirements are listed in `requirement.txt`. The supplied installer verifies Python 3.10 or newer, creates `.venv`, and installs all packages into that environment. From the artifact root, run:

```
./install_dependencies.sh
```

To use a different Python executable or virtual-environment location, set `PYTHON_BIN` or `VENV_DIR`, respectively, before running the installer.

Installing these packages requires Internet access once. Subsequent notebook runs use the data already included in the artifact.

3.2 Launching and running the notebook

Start Jupyter from the artifact root, which is the directory containing `plot.ipynb` and `results/`:

```
source .venv/bin/activate
jupyter notebook plot.ipynb
```

In the browser, select the `Python (Chronos Artifact)` kernel registered by the installer. This ensures that PyMuPDF and the other plotting dependencies are available. Run an individual cell to generate one figure, or select *Run All* to execute the complete notebook. The plotting programs use Matplotlib's non-interactive `Agg` backend, so output appears inline rather than in a separate desktop window.

Before each plot, the cell displays the paper figure number and the plotting script being executed. After the script completes, the cell renders the new PDF and prints its path below the image. A complete run produces 30 PDF files under `results/output/`. Re-running a cell replaces that cell's named output while leaving the source data unchanged.

The notebook can also be executed non-interactively:

```
source .venv/bin/activate
jupyter nbconvert plot.ipynb \
  --to notebook --execute \
  --output plot.executed.ipynb \
  --ExecutePreprocessor.timeout=300
find results/output -maxdepth 1 \
  -name '*.pdf' | wc -l
```

The expected PDF count is `30`. If a cell reports that it cannot find `results/`, stop Jupyter and launch it again from the artifact root. If a plotting script fails, the cell reports the script's exit code and relevant error message.

4 Supported functionality

The hosted demonstration checks the following properties:

1. the Powder/Emulab profile allocates the required node roles;
2. the provisioning scripts install the patched host/guest environment and create a working Kubernetes cluster;
3. the core, global slot coordinator, proxy, gNB, and UE containers start;
4. the gNB connects to the core and the UE completes registration;
5. the local and global coordinators advance the emulation through RAN slots; and
6. the attached UE obtains a working data plane and exchanges test traffic.

Passing these checks demonstrates that the artifact code is functional. The plotting notebook regenerates the paper figures from the supplied data, and the Powder scale sweep below reproduces the key relationship between emulation scale, available compute, and the dilation factor.

5 Access and evaluation window

The authors provide a running Chronos deployment on Powder for the artifact evaluation period from **September 1 through September 15**. The authors handle the Powder reservation, profile instantiation, kernel build, Kubernetes cluster, and repository dependencies so evaluators can begin directly with the functional checks.

To obtain access, each evaluator should post their **public SSH key** in a comment on the paper's HotCRP artifact-evaluation page. The authors will add the key to the Powder deployment and reply through HotCRP with the SSH command, host name, user name, and any required jump-host instructions. Only a public key (for example, the contents of `id_ed25519.pub`) should be posted. Evaluators must never send a private key, passphrase, password, or access token.

Access is available only during the stated September 1–15 window. Requests should be posted early enough to allow the authors to install the key and reply within the evaluation period. The evaluator needs an SSH client and the private key corresponding to the submitted public key.

The video tutorial `chronos-artifact.mp4` demonstrates the complete Powder evaluation workflow.

6 Evaluation on Powder/Emulab

6.1 Step 1: request and test SSH access

Post the evaluator's public SSH key in a HotCRP comment. After the authors reply with connection details, use the supplied command to log in. For example, the command will have the following general form (the actual values come from HotCRP):

```
ssh -i <private-key> <user>@<host>
```

Do not change SSH authorization files or share the connection details outside the artifact-evaluation process. If login fails, report the exact SSH error through HotCRP; do not post the private key.

6.2 Step 2: connect to the controller

The SSH command supplied through HotCRP connects to `node0`. Node 0 is the entry node for the experiment. From `node0`, connect to the controller and enter its Chronos deployment directory:

```
ssh ubuntu@10.2.1.2
cd chronos-auto-deploy
```

6.3 Step 3: configure and deploy one scale

This experiment reproduces a key paper result: as the number of emulated gNB/UE pairs increases on a fixed compute allocation, the work required per RAN slot eventually exceeds the available real-time compute budget. Chronos responds by increasing its dilation factor, allowing more wall-clock time per unit of virtual time while preserving coordinated RAN execution.

Evaluate the following gNB/UE pair counts in order:

```
1, 10, 50, 100, 200
```

For the first scale, edit `values.yaml`. Set `numberGNB` and `numberUE` to the same value. Deploy that configuration with:

```
./run_experiment.sh
```

When the interactive interface appears, press `p` to display pod status. Wait until all expected pods are running.

6.4 Step 4: verify connections and observe dilation

After all pods are running, execute the supplied core-log helper:

```
./core-logs.sh
```

Confirm from these logs that all configured gNBs and UEs are connected. In the interactive interface, enter `l globalsc` to display the global coordinator log. Observe and record the dilation value for the current scale. A value near 1 means that the emulation is keeping pace with real time; a value above 1 means Chronos is allocating additional wall-clock time to complete the virtual slots.

6.5 Step 5: tear down and repeat

Remove the current deployment before changing scale:

```
./teardown-experiment.sh
```

Wait for teardown to complete. Then edit `values.yaml`, set both `numberGNB` and `numberUE` to the next value in the sequence, and repeat Steps 3–5. Do not start a new deployment until the previous one has been removed.

The result is reproduced when the measurements show the same key trend as the paper: dilation remains close to real time while the fixed compute allocation has capacity, then increases as gNB/UE scale creates compute pressure. Exact values can vary with Powder hardware and load; the reproduced result is the increase in dilation that enables Chronos to cope with insufficient compute at higher scale while continuing coordinated slot execution. After recording the value for scale 200, run `./teardown-experiment.sh` once more and log out. Report failures, missing permissions, or questions through HotCRP.

7 Limitations and safety notes

- Cloud/testbed scheduling and network conditions may affect startup time and observed performance; evaluators can record these conditions when interpreting a live run.
- The deployment is shared during the evaluation window. Evaluators should avoid disruptive or privileged commands and report unhealthy components through HotCRP.

8 Summary of evaluation

The evaluator should inspect the supplied code, deployment scripts, plot outputs, and recorded data, then connect to the authors' running Powder deployment during September 1–15. Access requires only a public SSH key sent through HotCRP comments. The expected outcome is evidence that the supplied Chronos implementation executes end to end with working slot coordination, registration, and data-plane traffic. Evaluators also regenerate the paper plots from the supplied data and scripts and reproduce the key Powder result: as gNB/UE scale increases on fixed compute, Chronos increases its dilation factor to provide the wall-clock time required for coordinated slot execution.